Grado en Ingeniería Informática
Ingeniería Gráfica

Práctica 3. Modelos jerárquicos articulados



UNIVERSIDAD DE GRANADA

María Remacho Aneas
Grupo C2

Índice.

1. Introducción.....3
2. Grafo de escena.....3
3. Movimiento de las articulaciones.....4
    a. Piernas.....5
    b. Brazo.....7
4. Animación.....7
5. Controles de la práctica……..…..……..7

1. Introducción.

En esta práctica he abordado el diseño y la implementación de modelos jerárquicos articulados, que se usa en animación o por ejemplo, en construcción de objetos complejos.

Como hemos visto en clase, un modelo jerárquico se compone de múltiples partes conectadas entre sí, de modo que cada nodo acumula las transformaciones de sus antecesores.



Figura 1. Imagen de referencia.

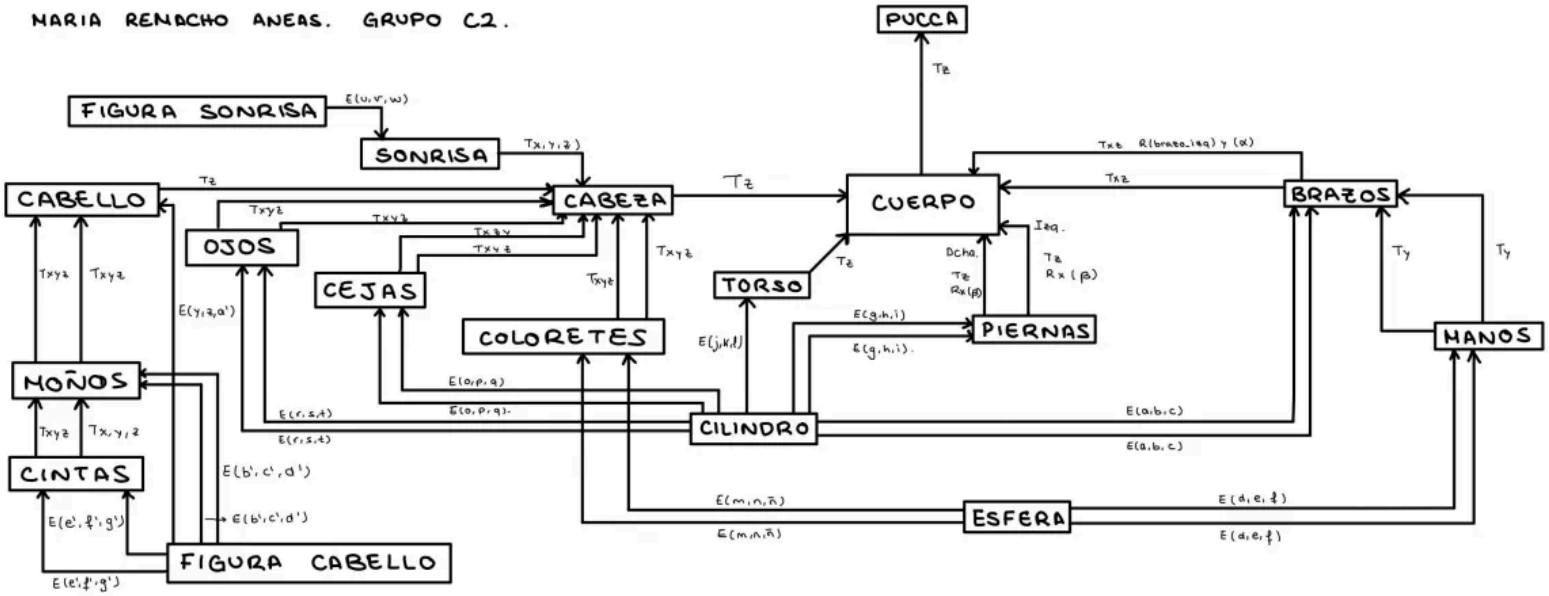
En mi caso he realizado al personaje de dibujos "Pucca" animado para que salude además de que pueda caminar hacia delante y hacia atrás.

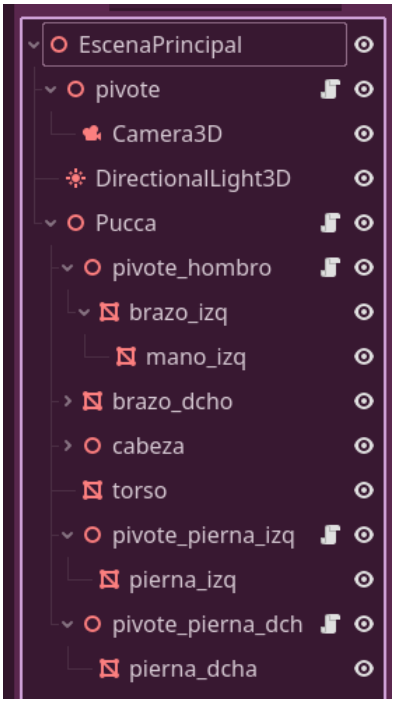
2. Grafo de escena.

El grafo de escena, como hemos aprendido, se trata de una estructura fundamental para poder organizar los elementos que componen nuestro modelo jerárquico articulado.
En este tipo de grado, cada parte del modelo se define como un nodo independiente y las relaciones entre los nodos padre e hijo indican cómo se hacen los movimientos de las articulaciones.

En mi figura, he construido el grafo de escena con todos los elementos que forman la figura aunque solo tres de ellos sean los que use la rotación. Estos son ambas piernas y uno de los brazos.

Las dos piernas, tanto la izquierda como la derecha tienen sus propias rotaciones, permitiendo que se muevan de manera alterna para simular el gesto de caminar cuando se pulsa el botón de traslación hacia delante o hacia atrás.





Aquí tengo todos los elementos que he organizado, que se componen mayormente por cilindros y esferas.
Además, el cabello, incluyendo los moños y las cintas los he podido obtener de la página https://sketchfab.com/feed importándo el documento .obj tal como hicimos en la práctica anterior.

3. Movimiento de las articulaciones.

He logrado la articulación en el modelo jerárquico aplicando transformaciones locales, en este caso, rotaciones, sobre unos nodos pivote para que pudiera rotar sobre esa zona y no rotara todo el objeto.

En cada articulación ha calculado cuántos grados de libertad tiene para que pudiera quedar un modelo más realista. Por ejemplo, en el brazo, lo he limitado para que no colisionara con la cabeza.

Para la práctica, he implementado las rotaciones en ambas piernas y uno de los brazos, haciendo que simule comportamientos como caminar y poder saludar al espectador.
Lo he hecho mediante scripts que controlan los límites de los ángulos y la dirección de movimiento.

a. Piernas.

Las piernas funcionan como dos articulaciones independientes, cada una con su propio nodo pivote y su propio script.
Para simular el movimiento de caminar, he hecho unas rotaciones en el eje X, haciendo que cada pierna alterne entre el ángulo mínimo y el máximo (-30º y 30º).

Además lo he sincronizado para que cuando una pierna avanza, la otra retroceda. Esto lo he conseguido poniendo direcciones iniciales opuestas.

Pierna Izquierda:

```
1   extends Node3D
2   @export var incremento := -0.75
3   var angulo := 0.0
4   var tope = false
5   var velocidad := 5
6
7   func _ready():
8       _process(incremento)
9
10  func _process(incremento):
11      if Input.is_action_pressed("hacia_delante"):
12          if not tope:
13              self.rotate_x(-incremento * velocidad)
14          else:
15              self.rotate_x(incremento * velocidad)
16
17          if self.rotation_degrees.x < -30:
18              tope = true
19          elif self.rotation_degrees.x > 30:
20              tope = false
21
22      if Input.is_action_pressed("hacia_atras"):
23          if not tope:
24              self.rotate_x(-incremento * velocidad)
25          else:
26              self.rotate_x(incremento * velocidad)
27
28          if self.rotation_degrees.x < -30:
29              tope = true
30          elif self.rotation_degrees.x > 30:
31              tope = false
32
```

Pierna derecha:

```
extends Node3D
@export var incremento := -0.75
var angulo := 0.0
var tope = false
var velocidad := 5

func _ready():
    _process(incremento)

func _process(incremento):
    if Input.is_action_pressed("hacia_delante"):
        if not tope:
            self.rotate_x(incremento * velocidad)
        else:
            self.rotate_x(-incremento * velocidad)

        if self.rotation_degrees.x > 30:
            tope = true
        elif self.rotation_degrees.x < -30:
            tope = false

    if Input.is_action_pressed("hacia_atras"):
        if not tope:
            self.rotate_x(incremento * velocidad)
        else:
            self.rotate_x(-incremento * velocidad)

        if self.rotation_degrees.x > 30:
            tope = true
        elif self.rotation_degrees.x < -30:
            tope = false

```

Además, cuento con una traslación del modelo completo cuando se “camina” para que se vea más real. Lo he implementado en el nodo general para que cuando se active, se muevan todos los componentes a la vez.

```
extends Node3D
@export var incremento := -0.75
@export var velocidad := 20.0

func _process(incremento):
    var direccion = Vector3.ZERO

    if Input.is_action_pressed("hacia_delante"):
        direccion.y -= 1

    if Input.is_action_pressed("hacia_atras"):
        direccion.y += 1

    if direccion != Vector3.ZERO:
        direccion = direccion.normalized()

    translate( direccion * velocidad * incremento )

```

b. Brazo.

El brazo izquierdo cuenta con un nodo propio que hace de pivote para controlar la rotación en el eje Y. Al igual que con las piernas, la rotación está limitada por dos valores, en este caso 0º y -50º, que determinan el movimiento que está permitido para que quede un movimiento natural y no colisiones con la cabeza del modelo.

```
1   extends Node3D
2   @export var activar := "brazo"
3   var tope = false
4   var velocidad := 9
5
6   var activa := true
7   var subiendo := false
8
9   func _process(delta):
10      if Input.is_action_just_pressed(activar):
11          activa = !activa
12
13      if not activa:
14          return
15
16      var inc = deg_to_rad(velocidad * delta)
17
18      if not tope:
19          self.rotate_y(-inc * velocidad)
20      else:
21          self.rotate_y(inc * velocidad)
22
23      if self.rotation_degrees.y < -50:
24          tope = true
25      elif self.rotation_degrees.y > 0:
26          tope = false
```

4. Animación.

Para la animación he implementado lo que venía en el guión de esta práctica. Es decir, hago que la rotación sea continua en cada frame.
Además, el nodo del brazo puede activarse y desactivarse con la tecla **D**, haciendo que se pueda ver mejor la articulación si es necesario.

5. Controles de la práctica.

Cuando comienza la ejecución podemos observar la animación del brazo en constante movimiento. Si queremos que finalice para poder rotarlo de manera “manual”, pulsamos la tecla **D.**

Luego, para poder caminar hacia delante o hacia atrás usaremos las teclas típicas de juego, **“W”** para poder avanzar hacia delante, y **“S”** para poder ir hacia atrás.

Por último, contamos con teclas de las flechas para todas las direcciones que nos permiten observar en el espacio al modelo y como realiza las rotaciones y animaciones así como las teclas de **“+”** y **“-”** que nos sirven para acercar y alejar el modelo en la vista.

```
extends Node3D
@export var incremento := 0.02
@export var trasladar := 5.0
@export var incremento2 := 1
@export var umbral := 0.02
var angulo := 0.0
var angulo2 := 0.0
var acciones := ["izquierda", "derecha","abajo", "arriba", "alejar", "acercar"]

func _ready():
	
	var delta = 0
	_process(delta)

func _process(delta):
	rotation=Vector3(0,0,0.0)
	position=Vector3(0,0,0.0)
	for action in acciones:
		if Input.is_action_pressed(action):
			match action:
				"izquierda": angulo2 -= incremento
				"derecha": angulo2 += incremento
				"arriba": angulo -= incremento
				"abajo": angulo += incremento*0.5
				"alejar": trasladar *= 1.05
				"acercar":
					trasladar -= incremento2
					if trasladar < umbral : trasladar = umbral
	rotate_y(angulo2)
	rotate_x(angulo)
	translate(Vector3(0, 0, trasladar))
```